

■ Tap Tamak

Backend Technical Specification
NestJS · Prisma · PostgreSQL

For use with Cursor IDE

1. Tech Stack & Setup

Layer	Technology	Purpose
Runtime	Node.js 20 LTS	Server runtime
Framework	NestJS 10	Modular backend framework with DI
ORM	Prisma 5	Type-safe DB access layer
Database	PostgreSQL 16	Primary relational database
Auth	JWT (access 15min + refresh 7d)	Stateless authentication
Validation	class-validator + class-transformer	DTO validation pipeline
File Upload	Multer + @aws-sdk/client-s3	Local dev → S3 in production
Push Notifs	Firebase Admin (FCM)	iOS + Android push
Real-time	Socket.io (@nestjs/websockets)	Order tracking + notifications
Docs	Swagger (@nestjs/swagger)	Auto-generated at /api/docs
Testing	Jest + Supertest	Unit + E2E tests

Bootstrap — main.ts

```
// src/main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.setGlobalPrefix('api/v1');
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
  app.useGlobalFilters(new HttpExceptionFilter());
  app.useGlobalInterceptors(new TransformInterceptor());
  app.enableCors({ origin: process.env.FRONTEND_URL });

  if (process.env.NODE_ENV !== 'production') {
    const config = new DocumentBuilder()
      .setTitle('Tap Tamak API').setVersion('1.0')
      .addBearerAuth().build();
    SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, config));
  }
  await app.listen(process.env.PORT ?? 3000);
}
```

.env required variables

```
DATABASE_URL="postgresql://user:pass@localhost:5432/taptamak"
JWT_ACCESS_SECRET="your-access-secret-min-32-chars"
JWT_REFRESH_SECRET="your-refresh-secret-min-32-chars"
FRONTEND_URL="http://localhost:4000"
STORAGE_TYPE="local" # "local" | "s3"
AWS_S3_BUCKET="taptamak-uploads"
AWS_REGION="ap-southeast-1"
FIREBASE_PROJECT_ID="taptamak-xxx"
GOOGLE_MAPS_API_KEY="AIza..."
HALYK_CLIENT_ID="..."
HALYK_CLIENT_SECRET="..."
```


2. Project Structure

```
tap-tamak-api/
├── prisma/
│   ├── schema.prisma          # Full data model
│   ├── migrations/
│   └── seed.ts
├──
├── src/
│   ├── core/                  # App-wide infrastructure
│   │   ├── auth/              # JWT guards, strategies, decorators
│   │   │   ├── guards/        # JwtAuthGuard, RolesGuard, JwtRefreshGuard
│   │   │   ├── strategies/    # AccessTokenStrategy, RefreshTokenStrategy
│   │   │   ├── decorators/    # @CurrentUser(), @Public(), @Roles()
│   │   │   ├── dto/
│   │   │   ├── auth.controller.ts
│   │   │   ├── auth.service.ts
│   │   │   └── auth.module.ts
│   │   ├── database/          # PrismaModule (@Global) + PrismaService
│   │   ├── config/            # app/jwt/s3/payment configs
│   │   └──
│   ├── common/                # Shared reusable code
│   │   ├── filters/           # HttpExceptionHandler
│   │   ├── interceptors/      # TransformInterceptor, LoggingInterceptor
│   │   ├── pipes/
│   │   ├── dto/               # PaginationDto
│   │   └── utils/             # haversine.ts, pagination.ts
│   ├──
│   ├── integrations/          # External service wrappers
│   │   ├── storage/           # LocalStorage + S3Storage (interface pattern)
│   │   ├── payment/           # HalykService, KaspiService, FreedomService
│   │   ├── google-maps/       # MapsService
│   │   └── firebase/          # FcmService
│   ├──
│   ├── modules/               # Domain feature modules
│   │   ├── user/
│   │   ├── cook/
│   │   ├── dish/
│   │   ├── order/             # includes order.gateway.ts (WebSocket)
│   │   ├── review/
│   │   ├── favorite/
│   │   ├── cart/
│   │   ├── payment/
│   │   ├── earning/
│   │   ├── statistics/
│   │   ├── notification/      # includes notification.gateway.ts (WebSocket)
│   │   └── upload/
│   ├──
│   ├── app.module.ts
│   └── main.ts
```

Auth Architecture Pattern

JwtAuthGuard is registered as a global APP_GUARD — all routes are protected by default. Use @Public() decorator to opt out. Use @Roles(Role.COOK) + RolesGuard for cook-only endpoints.

```
// Register guard globally in app.module.ts providers:
{ provide: APP_GUARD, useClass: JwtAuthGuard },
```


3. Database Schema (Prisma)

Key design decisions:

UUID PKs	All models use @id @default(uuid()) — no sequential IDs exposed in URLs
KZT as Int	All prices stored as plain integers (tenge). No decimal columns needed.
DB-backed Cart	Cart persists in DB (not session) — survives app restarts, works across devices
Order snapshots	OrderItem stores price+name at order time — history stays accurate after menu edits
Geo index	Cook model has @@index([latitude, longitude]) for nearby queries
Review constraint	orderId on Review is @unique — one review per order only

Enums

```
enum Role { USER, COOK, ADMIN }
enum VerificationStatus { PENDING, UNDER_REVIEW, APPROVED, REJECTED }
enum OrderStatus { PENDING, CONFIRMED, PREPARING, READY, ON_THE_WAY, DELIVERED, CANCELLED }
enum PaymentStatus { PENDING, PROCESSING, COMPLETED, FAILED, REFUNDED }
enum PaymentProvider { KASPI, HALYK, FREEDOM }
enum TransactionType { ORDER_PAYMENT, COOK_EARNING, PLATFORM_FEE, PAYOUT, REFUND }
enum NotificationType { ORDER_UPDATE, NEW_ORDER, REVIEW_RECEIVED, PAYMENT_UPDATE, PROMOTION, SYSTEM }
```

Core Models Overview

Model	Key Fields	Relations
User	id, email, phone, passwordHash, firstName, lastName, role, refreshToken	addresses[], orders[], reviews[], cart?
Cook	id, userId, businessName, bio, verificationStatus, latitude, longitude, isAvailable	orders[], reviews[], earnings[]
Dish	id, cookId, name, description, price(Int), imageUrl, category, tags	orders[], reviews[], favoritesBy[], cartItems[]
Order	id, orderNumber, userId, cookId, status, totalAmount, deliveryFee, deliveryAddress, paymentStatus, paymentProvider, paymentId	
OrderItem	id, orderId, dishId, quantity, price(snapshot), name(snapshot)	order, dish
Review	id, userId, cookId, orderId(@unique), dishId?, rating(1-5), comment	user, cook, order, dish?
Transaction	id, orderId?, cookId?, type, amount, status, provider, externalId, createdAt	order?, cook?
Cart	id, userId(@unique), cookId?	user, items[]
Address	id, userId, label, street, city, latitude, longitude, isDefault	user
Notification	id, userId, type, title, body, data(Json), isRead	user
FavoriteDish	userId, dishId @@unique([userId, dishId])	—
FavoriteCook	userId, cookId @@unique([userId, cookId])	—

Nearby Cooks Query (Haversine in raw SQL via Prisma)

```
// cook.service.ts
async findNearby(lat: number, lng: number, radius: number) {
  return this.prisma.$queryRaw`
    SELECT *, (
      6371 * acos(

```


4. API Endpoints Reference

Base URL: /api/v1 | Auth: Bearer JWT | All responses: { data, meta?, error? }

Auth

Method	Path	Access	Description
POST	/auth/register	PUBLIC	Register user (body: email, phone, password, firstName, lastName, role)
POST	/auth/login	PUBLIC	Login → returns { accessToken, refreshToken, user }
POST	/auth/refresh	AUTH	Rotate refresh token → returns new token pair
POST	/auth/logout	AUTH	Invalidate refresh token in DB
POST	/auth/forgot-password	PUBLIC	Send SMS/email reset link
POST	/auth/reset-password	PUBLIC	Reset password with token

Users

Method	Path	Access	Description
GET	/users/me	AUTH	Get current user profile
PATCH	/users/me	AUTH	Update name, avatar, phone
DELETE	/users/me	AUTH	Deactivate account
POST	/users/me/device-token	AUTH	Save FCM device token for push notifications
GET	/users/me/addresses	USER	List saved delivery addresses
POST	/users/me/addresses	USER	Create new address
PATCH	/users/me/addresses/:id	USER	Update address fields
DELETE	/users/me/addresses/:id	USER	Delete address

Cooks

Method	Path	Access	Description
GET	/cooks	PUBLIC	List cooks — query: ?cuisine=&minRating=&city=&page=&limit=
GET	/cooks/nearby	PUBLIC	Cooks near location — query: ?lat=&lng=&radius= (km)
GET	/cooks/:id	PUBLIC	Cook public profile with top dishes and recent reviews
GET	/cooks/me	COOK	Cook's own full profile + earnings summary
PATCH	/cooks/me	COOK	Update bio, specialties, schedule, minimumOrder, deliveryRadius
POST	/cooks/me/verification	COOK	Upload verification docs (multipart: identityDoc, healthCert, certificate?)
PATCH	/cooks/me/availability	COOK	Toggle isAvailable on/off
PATCH	/cooks/me/location	COOK	Update current lat/lng (called periodically from mobile)

Dishes

Method	Path	Access	Description
GET	/dishes	PUBLIC	Search dishes — query: ?category=&cookId=&tags=&minPrice=&maxPrice=
GET	/dishes/:id	PUBLIC	Full dish detail including cook info and avg rating
GET	/cooks/:cookId/dishes	PUBLIC	All dishes for a cook, grouped by category
POST	/dishes	COOK	Create dish (name, description, price, category, tags, imageUrl)
PATCH	/dishes/:id	COOK	Update dish — owner only
DELETE	/dishes/:id	COOK	Delete dish — owner only
PATCH	/dishes/:id/availability	COOK	Toggle dish isAvailable

Orders

Method	Path	Access	Description
POST	/orders	USER	Create order from current cart (validates items, calculates totals, clears cart)
GET	/orders	AUTH	List orders — USER sees own, COOK sees received. Query: ?status=&page=
GET	/orders/:id	AUTH	Order detail with items, status history, delivery address
PATCH	/orders/:id/status	COOK	Advance status: CONFIRMED→PREPARING→READY→ON_THE_WAY→DELIVERED
POST	/orders/:id/cancel	AUTH	Cancel order. USER: only before PREPARING. COOK: before DELIVERED.

Reviews

Method	Path	Access	Description
POST	/orders/:orderId/review	USER	Create review (rating 1–5, comment, photoUrls[]). One per order.
GET	/cooks/:cookId/reviews	PUBLIC	Paginated reviews for cook. Query: ?rating=&hasPhoto=&sort=newest
GET	/dishes/:dishId/reviews	PUBLIC	Reviews for a specific dish
POST	/reviews/:id/reply	COOK	Cook replies to a review. Only cook who owns the order can reply.
PATCH	/reviews/:id/reply	COOK	Edit existing reply

Favorites

Method	Path	Access	Description
GET	/favorites/dishes	USER	List favorited dishes with cook info
POST	/favorites/dishes/:dishId	USER	Add dish to favorites (idempotent)
DELETE	/favorites/dishes/:dishId	USER	Remove dish from favorites
GET	/favorites/cooks	USER	List favorited cooks
POST	/favorites/cooks/:cookId	USER	Add cook to favorites (idempotent)
DELETE	/favorites/cooks/:cookId	USER	Remove cook from favorites

Cart

Method	Path	Access	Description
GET	/cart	USER	Get cart with items, totals, deliveryFee estimate, cookId
POST	/cart/items	USER	Add item. If new cook → confirm cart clear first (handle on frontend)
PATCH	/cart/items/:dishId	USER	Update quantity. quantity=0 removes item.
DELETE	/cart/items/:dishId	USER	Remove single item
DELETE	/cart	USER	Clear entire cart

Payment

Method	Path	Access	Description
POST	/payments/initiate	USER	Start payment — body: { orderId, provider: HALYK KASPI FREEDOM } → return
GET	/payments/:id/status	AUTH	Poll payment status
POST	/payments/webhook/halyk	PUBLIC	Halyk ePay postLink callback — validates token, updates order+transaction
POST	/payments/webhook/freedom	PUBLIC	Freedom Pay result_url callback — verifies MD5 sig
POST	/payments/webhook/kaspi	PUBLIC	Kaspi/AiPay callback — IP whitelist + API key auth

Earnings & Statistics (Cook-side)

Method	Path	Access	Description
GET	/earnings	COOK	Summary: totalEarned, pendingPayout, totalPaid, commissionPaid
GET	/earnings/transactions	COOK	Paginated transactions. Query: ?type=&startDate=&endDate=&status=
POST	/earnings/payout	COOK	Request payout to bank/Kaspi. Minimum payout threshold enforced.
GET	/statistics/overview	COOK	Dashboard: todayOrders, weekRevenue, monthRevenue, avgRating
GET	/statistics/top-dishes	COOK	Top dishes by orderCount and revenue. Query: ?period=7d 30d 90d
GET	/statistics/peak-hours	COOK	24-element array: hourly order count for last 30 days
GET	/statistics/revenue-chart	COOK	Daily revenue array. Query: ?period=7d 30d 90d&metric=revenue orders avg

Upload & Notifications

Method	Path	Access	Description
POST	/upload/image	AUTH	Multipart. Max 5MB. JPEG/PNG/WebP. Returns { url }
POST	/upload/document	COOK	Multipart. Max 10MB. JPEG/PNG/PDF. Returns { url }
GET	/notifications	AUTH	Paginated notifications, unread first
PATCH	/notifications/:id/read	AUTH	Mark one notification as read
POST	/notifications/read-all	AUTH	Mark all as read
GET	/notifications/unread-count	AUTH	Returns { count: number }

5. Payment Integration

Halyk ePay (Primary — implement first)

Covers 65%+ of Kazakh online transactions. Supports Visa, Mastercard, Apple Pay, Google Pay, Halyk QR.

```
// integrations/payment/halyk.service.ts
// Step 1: Get OAuth2 token
POST https://epay-oauth.homebank.kz/oauth2/token
Body: { grant_type, client_id, client_secret, terminal_id, invoice_id, amount, currency: 'KZT', ... }

// Step 2: Frontend renders Halyk payment form
halyk.pay({ invoiceId, backLink, failureBackLink, postLink, language: 'RU', terminal, accessToken })

// Step 3: Halyk POSTs to your postLink webhook
POST /api/v1/payments/webhook/halyk
// Body includes: code ('ok'|'error'), reference, cardMask, amount, invoiceId
// Validate token, update order.paymentStatus, create Transaction records

// Test env: https://test-epay.epayment.kz
// Docs: https://epayment.kz/docs/platzezhnaya-stranica
```

Kaspi.kz (via AiPay wrapper)

14M+ Kaspi users in Kazakhstan. No direct API — use AiPay.kz wrapper.

```
// integrations/payment/kaspi.service.ts
// Step 1: Create invoice via AiPay
POST https://api.aipay.kz/v1/invoices
Headers: { Authorization: 'Bearer AIPAY_API_KEY' }
Body: { amount, currency: 'KZT', phone: customerKaspiPhone, description, callback_url, success_url }
Response: { invoice_id, payment_url } // redirect customer to payment_url

// Step 2: Customer opens Kaspi app and confirms payment

// Step 3: AiPay POSTs to your callback_url
POST /api/v1/payments/webhook/kaspi
// Verify with IP whitelist + AiPay shared secret header
// Update order.paymentStatus on 'success' status
```

Freedom Pay

```
// integrations/payment/freedom.service.ts
// Step 1: Init payment
POST https://api.freedompay.kz/init_payment.php
Body: { pg_merchant_id, pg_order_id, pg_amount, pg_currency: 'KZT',
        pg_description, pg_salt, pg_sig (MD5), pg_result_url, pg_success_url, pg_failure_url }
Response: { pg_status, pg_payment_id, pg_redirect_url }

// Step 2: Redirect customer to pg_redirect_url

// Step 3: Freedom POSTs to pg_result_url (retries every 30min for 2hrs if down)
POST /api/v1/payments/webhook/freedom
// Verify pg_sig (MD5 of sorted params + secret)
// Respond with XML: <response><pg_status>ok</pg_status></response>
```

Webhook Idempotency Pattern

```
// Always check before processing to prevent double-processing
async handleWebhook(externalPaymentId: string, status: 'success' | 'failure') {
  const existing = await this.prisma.transaction.findUnique({
    where: { externalId: externalPaymentId }
  });
  if (existing?.status === PaymentStatus.COMPLETED) return; // Already processed

  await this.prisma.$transaction([
    this.prisma.order.update({ where: { paymentId: externalPaymentId },
      data: { paymentStatus: status === 'success' ? 'COMPLETED' : 'FAILED' } }),
    this.prisma.transaction.create({ data: { type: 'ORDER_PAYMENT', status: 'COMPLETED', ... } }),
  ]);
}
```

6. Real-time (WebSocket Gateways)

```
// modules/order/order.gateway.ts
@WebSocketGateway({ namespace: '/orders', cors: true })
export class OrderGateway {
  @WebSocketServer() server: Server;

  // Broadcast order status change to customer + cook
  notifyStatusChange(orderId: string, payload: OrderStatusPayload) {
    this.server.to(`order:${orderId}`).emit('status_changed', payload);
  }

  // Broadcast courier location update for delivery tracking
  updateCourierLocation(orderId: string, lat: number, lng: number) {
    this.server.to(`order:${orderId}`).emit('location_update', { lat, lng });
  }

  @SubscribeMessage('join_order')
  handleJoin(@MessageBody() orderId: string, @ConnectedSocket() client: Socket) {
    client.join(`order:${orderId}`);
  }
}

// modules/notification/notification.gateway.ts
@WebSocketGateway({ namespace: '/notifications', cors: true })
export class NotificationGateway {
  @WebSocketServer() server: Server;

  notifyUser(userId: string, notification: NotificationPayload) {
    this.server.to(`user:${userId}`).emit('notification', notification);
  }

  @SubscribeMessage('join')
  handleJoin(@MessageBody() userId: string, @ConnectedSocket() client: Socket) {
    // Validate JWT token from handshake auth
    client.join(`user:${userId}`);
  }
}
```

7. Rating Recalculation

Update cook and dish ratings after every new review using a Prisma transaction:

```
// review.service.ts – after creating a review
async recalculateCookRating(cookId: string) {
  const result = await this.prisma.review.aggregate({
    where: { cookId },
    _avg: { rating: true },
    _count: { rating: true },
  });
  await this.prisma.cook.update({
    where: { id: cookId },
    data: {
      rating: Math.round((result._avg.rating ?? 0) * 10) / 10,
      totalReviews: result._count.rating,
    },
  });
}
```

MVP Critical Path

Week	Focus
1–2	Prisma schema + migrations, Auth module (register/login/refresh), PrismaModule global setup
3–4	Cook module (profile, verification, location), Dish module (CRUD, search), Upload module
5–6	Cart module, Order module (create, status flow), Order WebSocket gateway
7–8	Halyk ePay payment integration + webhook, Transaction + Earning modules
9–10	Reviews module (create + reply + rating recalc), Favorites module, Notification module + FCM
11–12	Statistics module, Kaspi + Freedom Pay, Address module, Polish + Swagger docs